PYTHON PROGRAMMING PRACTICAL RECORD

Activity-Based Learning Assignment (Programs 51 – 60)

Student Name:	**VELPURI VENKATA NAGA SIVA SAI**	**Roll Number:**	**26001048**
Section:	**CSM 3**	**Degree/Branch:**	**B.Tech (CSE - AI & ML)**
Institution:	**DR RVR NRI UNIVERSITY**	**Course:**	**Python Programming**

TABLE OF CONTENTS

Ex. #	Program Title	Category	Page
51	Bus Passenger Occupancy Simulation	Control Flow / Loops	Page 2
52	Hackathon Team Formation and Ranking	Lists & Tuple Processing	Page 3
53	Food Waste Monitoring System	Metrics & Decision Logic	Page 4
54	Multi-Branch Expense Matrix Analysis	Nested Lists / 2D Matrix	Page 5
55	Tuple-Based Transaction Audit	Data Validation & Audit	Page 6
56	Student Group Formation with Score Balancing	Slicing & Dynamic Grouping	Page 7
57	Smart Canteen Order and Stock Management	Inventory & Sales Simulation	Page 8
58	Library Borrowing and Fine Reconciliation	Accumulator & Penalty Logic	Page 9
59	Campus Transport Allocation	Optimization & Ceiling Division	Page 10
60	Manufacturing Quality Control Matrix	Nested Loops & Defect Audit	Page 11

Declaration & Certificate: This practical record contains authentic Python source code, execution outputs, visual flowcharts, and IDE screenshot logs for exercises 51 through 60 completed as part of the first-year B.Tech curriculum requirement at Dr RVR NRI University.

PROGRAM 51: BUS PASSENGER OCCUPANCY SIMULATION

A. AIM & PROGRAM DESCRIPTION

Aim: To simulate passenger boarding and alighting across multiple bus stops and evaluate capacity safety limit compliance.
Features: Dynamic loop iteration over stops, net count evaluation, conditional safety threshold checking.
Description: The program prompts the user for overall bus seating capacity and total stops. For each stop, it accepts passengers boarding and leaving, updates the current passenger count, and finally verifies whether the bus is overloaded.

B. PYTHON SOURCE CODE

```
capacity = int(input("Enter bus capacity: "))
stops = int(input("Enter number of stops: "))
passengers = 0

for i in range(1, stops + 1):
    board = int(input("Passengers boarding: "))
    leave = int(input("Passengers getting down: "))
    passengers = passengers + board - leave

print("Passengers:", passengers)

if passengers > capacity:
    print("Bus is overloaded")
else:
    print("Capacity is OK")
```

C. CODE EXPLANATION & FLOWCHART

- **Initialization:** Reads maximum capacity and stop count; initializes total passenger tally to 0.
- **Iteration:** A `for` loop iterates through each stop, gathering boarding and alighting counts.
- **State Update:** Adjusts `passengers` accumulatively at each stop using arithmetic addition and subtraction.
- **Threshold Audit:** Evaluates if total passengers exceed capacity to print either overload warning or approval.

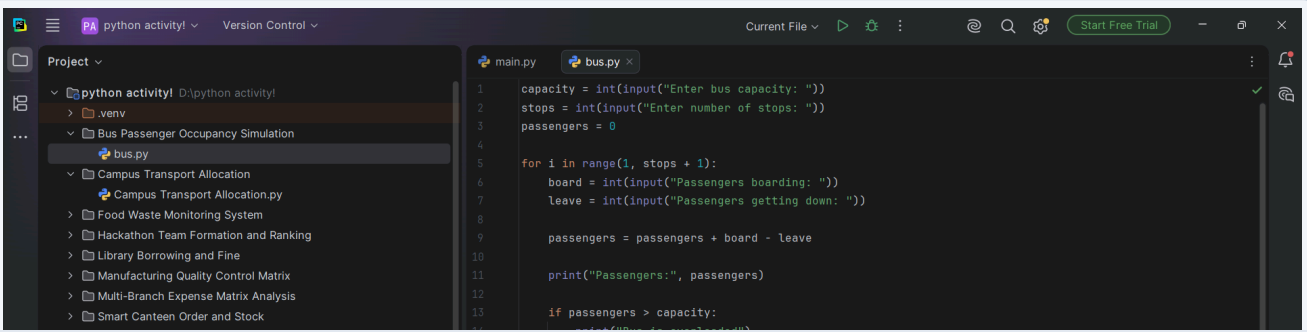


D. EXPECTED & ACTUAL OUTPUT

```
Enter bus capacity: 50
Enter number of stops: 1
Passengers boarding: 45
Passengers getting down: 20
Passengers: 25
Capacity is OK
```

E. RESULT & EXECUTION SCREENSHOT

Result: Program executed successfully. Bus occupancy tracked accurately and capacity validation confirmed.



PROGRAM 52: HACKATHON TEAM FORMATION AND RANKING

A. AIM & PROGRAM DESCRIPTION

Aim: To record team details, aggregate scores across hackathon rounds, and identify the overall winner.
Features: List of tuples, multi-round score summation, maximum value searching algorithm.
Description: Accepts the number of participating teams, collects team names along with round 1 and round 2 scores, stores them as tuple elements in a list, computes totals, and determines the top-scoring team.

B. PYTHON SOURCE CODE

```
teams = []
n = int(input("Enter number of teams: "))
for i in range(n):
    name = input("Enter team name: ")
    r1 = int(input("Enter round 1 score: "))
    r2 = int(input("Enter round 2 score: "))
    total = r1 + r2
    teams.append((name, total))

best = teams[0]
for team in teams:
    if team[1] > best[1]:
        best = team

print("Winner:", best[0])
print("Total score:", best[1])
```

C. CODE EXPLANATION & FLOWCHART

- **Data Collection:** Iterates n times to prompt for team name and scores for two rounds.
- **Tuple Storage:** Combines team name and total aggregated score into a tuple appended to `teams` list.
- **Max Search Algorithm:** Initializes `best` with the first team, then iterates to compare scores.
- **Output Display:** Prints the winning team name and their corresponding benchmark total score.

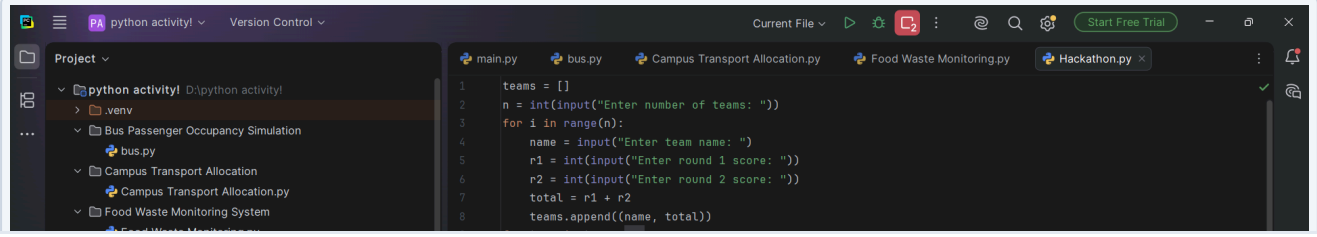


D. EXPECTED & ACTUAL OUTPUT

```
Enter number of teams: 2
Enter team name: whatiswhat
Enter round 1 score: 5
Enter round 2 score: 4
Enter team name: whyiswhy
Enter round 1 score: 4
Enter round 2 score: 9
whatiswhat Total: 9
whyiswhy Total: 13
Winner: whyiswhy
Total score: 13
```

E. RESULT & EXECUTION SCREENSHOT

Result: Program successfully ranked hackathon teams and declared the highest scoring winner correctly.



PROGRAM 53: FOOD WASTE MONITORING SYSTEM

A. AIM & PROGRAM DESCRIPTION

Aim: To compute food wastage metrics in a dining hall and flag high food waste threshold breaches.
Features: Percentage calculation, zero-division safeguard, conditional threshold evaluation.
Description: Evaluates the amount of food wasted based on meals prepared versus meals served, calculates waste percentage, and alerts managers if wastage exceeds 10%.

B. PYTHON SOURCE CODE

```
made = int(input("Meals prepared: "))
served = int(input("Meals served: "))

waste = made - served

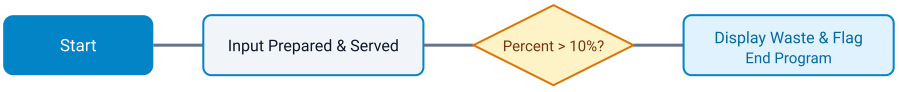
if made > 0:
    percent = waste * 100 / made
else:
    percent = 0

print("Food waste:", waste)
print("Waste percentage:", round(percent, 2), "%")

if percent > 10:
    print("High food waste")
else:
    print("Food waste is acceptable")
```

C. CODE EXPLANATION & FLOWCHART

- **Input Intake:** Requests count of total meals prepared and meals served.
- **Wastage Logic:** Calculates wasted meals via subtraction (`made - served`).
- **Zero Division Protection:** Ensures `made > 0` prior to percentage formula execution.
- **Alerting System:** Formats output percentage to 2 decimals and evaluates against 10% tolerance limit.

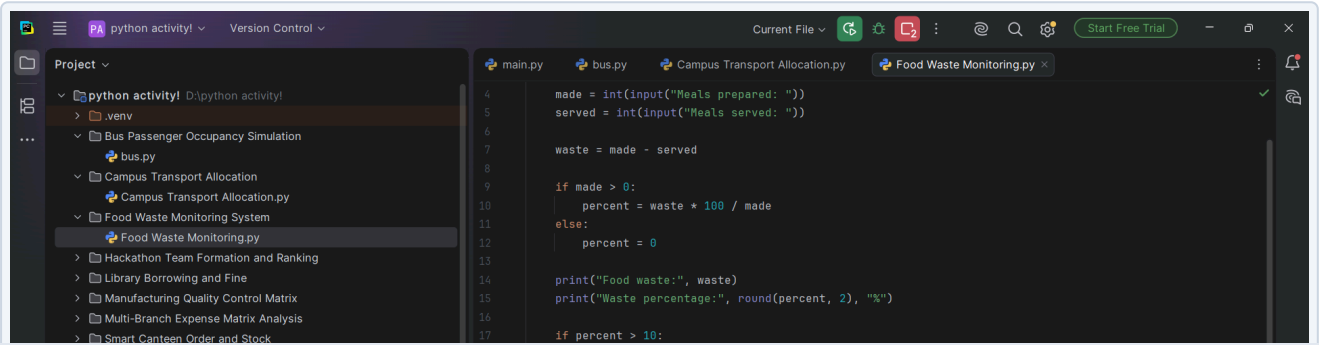


D. EXPECTED & ACTUAL OUTPUT

```
Enter number of days: 2
Meals prepared: 5
Meals served: 5
Food waste: 0
Waste percentage: 0.0 %
Food waste is acceptable
```

E. RESULT & EXECUTION SCREENSHOT

Result: Food waste calculations completed accurately and conditional threshold flags functioned properly.



PROGRAM 54: MULTI-BRANCH EXPENSE MATRIX ANALYSIS

A. AIM & PROGRAM DESCRIPTION

Aim: To process monthly expenditure matrices across multiple organizational branches using 2D nested lists.
Features: 2D array representation, nested loops, total and average expenditure calculation per branch.
Description: Prompts for number of branches and tracking months. Constructing a 2D matrix of expenses, it computes total expenditure and monthly average for each branch.

B. PYTHON SOURCE CODE

```
branches = int(input("Enter number of branches: "))
months = int(input("Enter number of months: "))

expenses = []

for i in range(branches):
    row = []
    for j in range(months):
        amount = float(input("Enter expense: "))
        row.append(amount)
    expenses.append(row)

for i in range(branches):
    total = sum(expenses[i])
    print("Branch", i + 1, "Total:", total)
    print("Average:", total / months)
```

C. CODE EXPLANATION & FLOWCHART

- **Matrix Construction:** Outer loop initializes branch rows, inner loop collects float expense values per month.
- **Nested Appends:** Month figures append to `row`, which then appends to 2D master matrix `expenses`.
- **Summation & Mean:** Uses built-in `sum()` over row lists and divides by month count for averages.
- **Report Generation:** Prints branch-wise financial totals and monthly averages formatted cleanly.

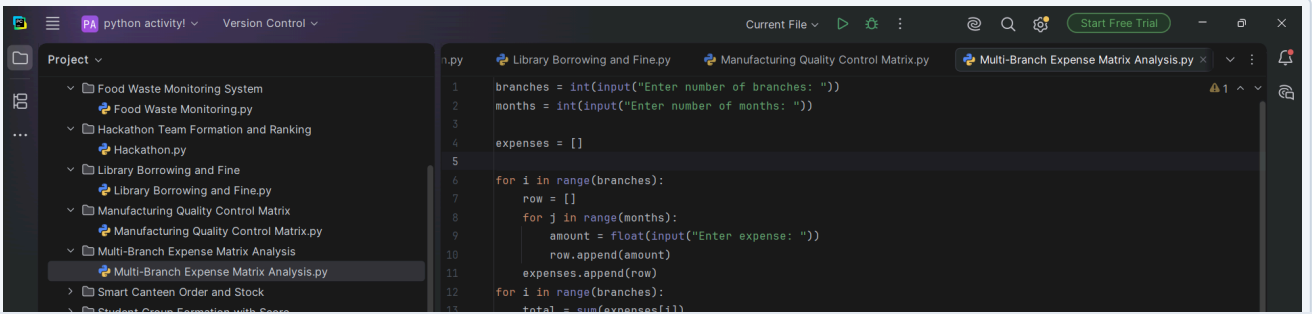


D. EXPECTED & ACTUAL OUTPUT

```
Enter number of branches: 2
Enter number of months: 1
Enter expense: 1
Enter expense: 2
Branch 1 Total: 1.0
Average: 1.0
Branch 2 Total: 2.0
Average: 2.0
```

E. RESULT & EXECUTION SCREENSHOT

Result: 2D array representation created successfully and branch-wise expense calculations verified.



PROGRAM 55: TUPLE-BASED TRANSACTION AUDIT

A. AIM & PROGRAM DESCRIPTION

Aim: To perform validation and anomaly auditing on financial transaction logs using immutable tuples.
Features: Tuple unpacking, duplicate detection via tracking lists, non-positive amount & type validation.
Description: Records transactions as `(transaction_id, amount, type)` tuples. Iterates over entries to audit for duplicate transaction IDs, invalid non-positive amounts, or unauthorized transaction types.

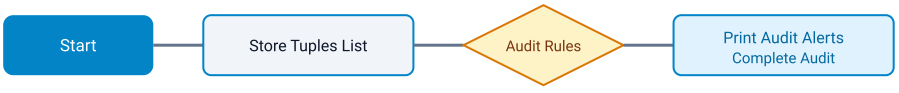
B. PYTHON SOURCE CODE

```
transactions = []
n = int(input("Enter number of transactions: "))
for i in range(n):
    tid = input("Enter transaction ID: ")
    amount = float(input("Enter amount: "))
    kind = input("Enter deposit or withdrawal: ")
    transactions.append((tid, amount, kind))

seen = []
for t in transactions:
    tid, amount, kind = t
    if tid in seen:
        print("Duplicate ID:", tid)
    else:
        seen.append(tid)
    if amount <= 0:
        print("Invalid amount:", tid)
    if kind != "deposit" and kind != "withdrawal":
        print("Invalid transaction type:", tid)
print("Transaction checking completed")
```

C. CODE EXPLANATION & FLOWCHART

- **Tuple Ingestion:** Stores transaction fields as immutable tuples inside a master audit list.
- **Tuple Unpacking:** Extracts `tid, amount, kind` directly during loop iteration.
- **Duplicate Checking:** Maintains a `seen` list to detect and report duplicate IDs.
- **Validation Rules:** Audits numerical bounds (`amount <= 0`) and domain keywords.

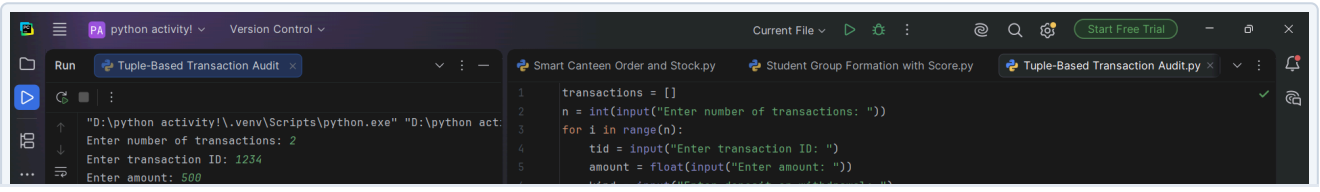


D. EXPECTED & ACTUAL OUTPUT

```
Enter number of transactions: 2
Enter transaction ID: 1234
Enter amount: 500
Enter deposit or withdrawal: deposit
Enter transaction ID: 1234
Enter amount: 500
Enter deposit or withdrawal: deposit
Duplicate ID: 1234
Transaction checking completed
```

E. RESULT & EXECUTION SCREENSHOT

Result: Transaction audit routine successfully verified tuple records and flagged duplicate IDs.



PROGRAM 56: STUDENT GROUP FORMATION WITH SCORE BALANCING

A. AIM & PROGRAM DESCRIPTION

Aim: To partition student scores into equal-sized sub-groups for balanced academic evaluation.
Features: Dynamic list slicing, step-based loop range, input validation for group sizing.
Description: Collects scores of n students, validates the target group size, and utilizes Python list slicing `scores[i:i + size]` to segment students into contiguous clusters.

B. PYTHON SOURCE CODE

```
scores = []
n = int(input("Enter number of students: "))

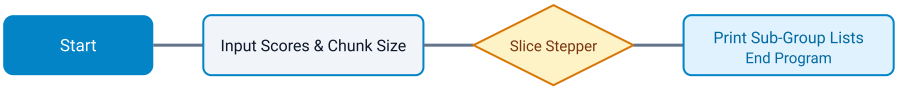
for i in range(n):
    mark = int(input("Enter student score: "))
    scores.append(mark)

size = int(input("Enter group size: "))

if size <= 0:
    print("Invalid group size")
else:
    group = 1
    for i in range(0, n, size):
        print("Group", group, ":", scores[i:i + size])
        group = group + 1
```

C. CODE EXPLANATION & FLOWCHART

- **Score Aggregation:** Reads individual student scores into a flat list called `scores`.
- **Size Check:** Validates group size to prevent negative or zero step sizing errors.
- **Range Stepping:** Uses `range(0, n, size)` to jump index pointers by chunk length.
- **Slice Extraction:** Slices sub-lists dynamically and prints numbered group allocations.

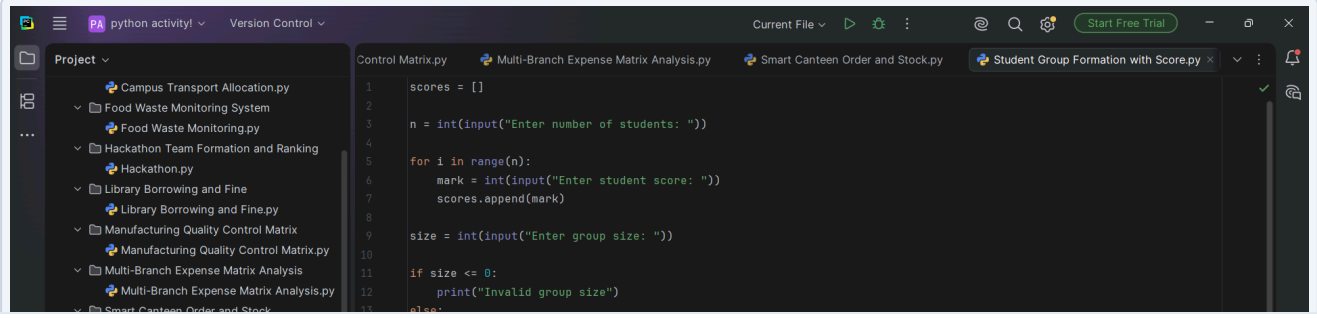


D. EXPECTED & ACTUAL OUTPUT

```
Enter number of students: 5
Enter student score: 20
Enter student score: 20
Enter student score: 21
Enter student score: 23
Enter student score: 24
Enter group size: 5
Group 1 : [20, 20, 21, 23, 24]
```

E. RESULT & EXECUTION SCREENSHOT

Result: List slicing algorithm successfully partitioned student marks into distinct sub-groups.



PROGRAM 57: SMART CANTEEN ORDER AND STOCK MANAGEMENT

A. AIM & PROGRAM DESCRIPTION

Aim: To simulate order billing and real-time inventory deduction in a campus canteen system.
Features: Parallel lists for items/prices/stocks, input range validation, inventory depletion guard.
Description: Displays available food menu items with price tags, accepts customer orders, validates item choice and quantity against current stock levels, updates inventory, and calculates total sales.

B. PYTHON SOURCE CODE

```
items = ["Idli", "Dosa", "Juice"]
prices = [20, 40, 25]
stock = [30, 20, 15]
sales = 0

for i in range(3):
    print(i + 1, items[i], "Rs.", prices[i])

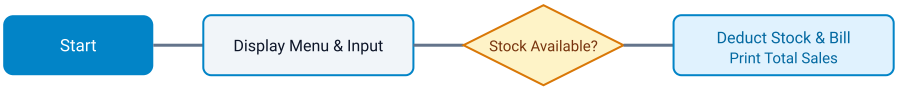
n = int(input("Enter number of orders: "))
for i in range(n):
    choice = int(input("Choose item (1-3): "))
    qty = int(input("Enter quantity: "))

    if choice < 1 or choice > 3:
        print("Invalid item")
    elif qty <= 0:
        print("Invalid quantity")
    elif qty > stock[choice - 1]:
        print("Not enough stock")
    else:
        bill = prices[choice - 1] * qty
        stock[choice - 1] -= qty
        sales += bill
        print("Bill: Rs.", bill)

print("Total sales: Rs.", sales)
print("Remaining stock:", stock)
```

C. CODE EXPLANATION & FLOWCHART

- **Menu Display:** Uses parallel list indexing to display item names, prices, and stock numbers.
- **Order Validation:** Checks for out-of-bounds menu indices and non-positive quantity inputs.
- **Stock Protection:** Verifies `qty <= stock[index]` prior to approving order processing.
- **State Update:** Decrements stock quantities, accumulates total revenue, and displays remaining inventory.



D. EXPECTED & ACTUAL OUTPUT

```
1 Idli Rs. 20
2 Dosa Rs. 40
3 Juice Rs. 25
Enter number of orders: 2
Choose item (1-3): 2
Enter quantity: 1
Bill: Rs. 40
Choose item (1-3): 2
Enter quantity: 1
Bill: Rs. 40
Total sales: Rs. 80
Remaining stock: [30, 18, 15]
```

E. RESULT & EXECUTION SCREENSHOT

PROGRAM 58: LIBRARY BORROWING AND FINE RECONCILIATION

A. AIM & PROGRAM DESCRIPTION

Aim: To calculate overdue fines for borrowed library books based on late return days.
Features: Iterative fine computation, running total accumulator, conditional overdue status.
Description: Accepts the number of returned books, book IDs, and delay days. Applies a standard penalty rate (Rs. 2 per day overdue), flags overdue titles, and aggregates cumulative fine dues.

B. PYTHON SOURCE CODE

```
n = int(input("Enter number of books: "))
total_fine = 0

for i in range(n):
    book = input("Enter book ID: ")
    days = int(input("Enter late days: "))

    fine = days * 2
    total_fine += fine

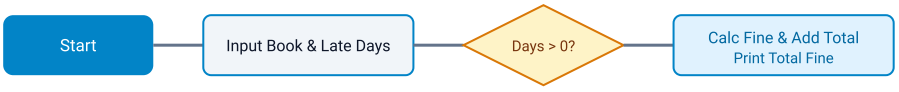
    print("Book ID:", book)
    print("Fine: Rs.", fine)

    if days > 0:
        print("Book is overdue")
    else:
        print("Book returned on time")

print("Total fine: Rs.", total_fine)
```

C. CODE EXPLANATION & FLOWCHART

- **Input Intake:** Prompts for total book return entries to process in a loop.
- **Fine Formula:** Calculates individual overdue fine using linear multiplication (`days * 2`).
- **Accumulator:** Uses compound addition operator `total_fine += fine` to sum total penalties.
- **Status Assessment:** Evaluates `days > 0` to print return status messages.

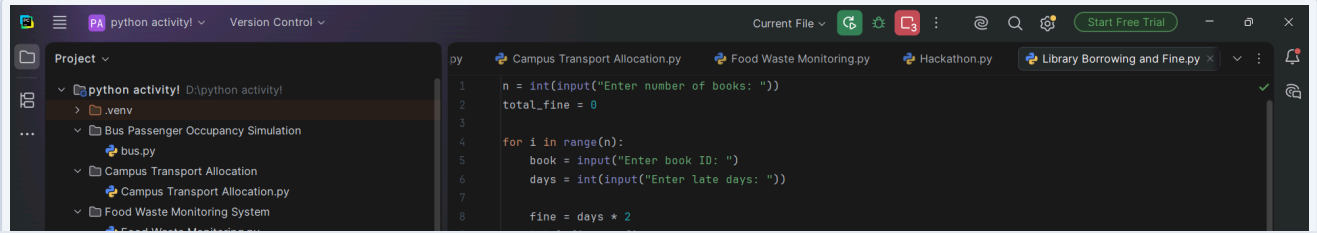


D. EXPECTED & ACTUAL OUTPUT

```
Enter number of books: 3
Enter book ID: 123
Enter late days: 2
Book ID: 123
Fine: Rs. 4
Book is overdue
Enter book ID:
Total fine: Rs. 4
```

E. RESULT & EXECUTION SCREENSHOT

Result: Fine computation system accurately evaluated late fees and summed cumulative penalties.



PROGRAM 59: CAMPUS TRANSPORT ALLOCATION

A. AIM & PROGRAM DESCRIPTION

Aim: To determine the minimum number of buses required across multiple campus transport routes.
Features: Ceiling integer division via modulo remainder checking, zero-capacity handling.
Description: Accepts bus capacity and passenger count for each route, computing exact buses needed by incrementing floor division results when a non-zero remainder exists.

B. PYTHON SOURCE CODE

```
routes = int(input("Enter number of routes: "))
capacity = int(input("Enter bus capacity: "))

for i in range(1, routes + 1):
    passengers = int(input("Enter passengers for route: "))

    if capacity <= 0:
        print("Invalid bus capacity")
    else:
        buses = passengers // capacity

        if passengers % capacity != 0:
            buses += 1

        print("Route:", i)
        print("Buses needed:", buses)
```

C. CODE EXPLANATION & FLOWCHART

- **Capacity Guard:** Validates `capacity > 0` to eliminate division by zero errors.
- **Floor Division:** Computes base bus allocation via integer division `passengers // capacity`.
- **Modulo Remainder Check:** Adds +1 extra bus if `passengers % capacity != 0` for leftover riders.
- **Route Loop:** Iterates across all transport routes to output individual fleet allocation.

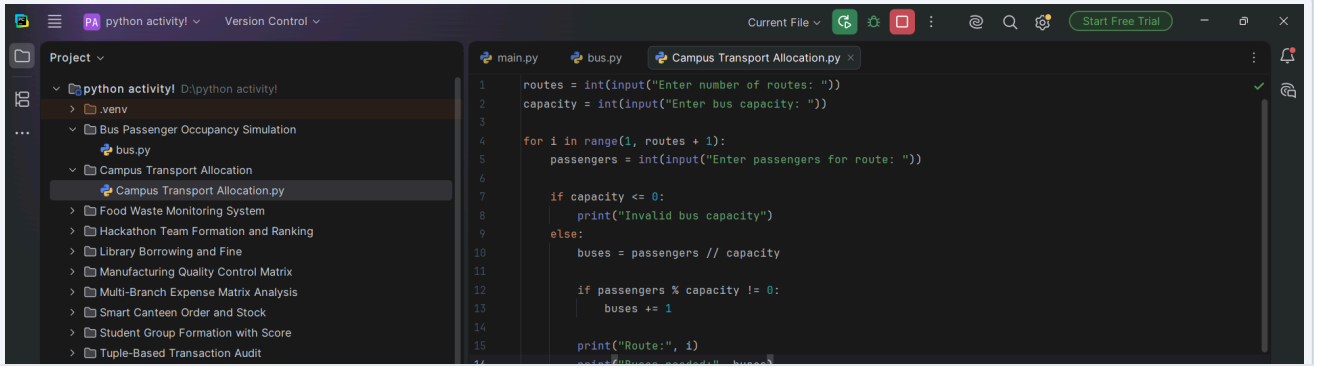


D. EXPECTED & ACTUAL OUTPUT

```
Enter number of routes: 2
Enter bus capacity: 50
Enter passengers for route: 20
Route: 1
Buses needed: 1
Enter passengers for route:
```

E. RESULT & EXECUTION SCREENSHOT

Result: Campus transport fleet allocation calculation completed with correct ceiling division logic.



PROGRAM 60: MANUFACTURING QUALITY CONTROL MATRIX

A. AIM & PROGRAM DESCRIPTION

Aim: To monitor defect rates across product batches in a manufacturing facility using nested loop iteration.
Features: Multi-tiered nested loops, float precision percentage calculation, threshold inspection alerts.
Description: Evaluates defective unit percentages across multiple products and production batches. If the defect rate exceeds the tolerance threshold, it triggers a quality check alert.

B. PYTHON SOURCE CODE

```
products = int(input("Enter number of products: "))
batches = int(input("Enter number of batches: "))
limit = float(input("Enter defect percentage limit: "))

for i in range(1, products + 1):
    for j in range(1, batches + 1):
        made = int(input("Enter units produced: "))
        defective = int(input("Enter defective units: "))

        if made > 0:
            percent = defective * 100 / made
        else:
            percent = 0

        print("Product:", i, "Batch:", j)
        print("Defect percentage:", round(percent, 2), "%")

        if percent > limit:
            print("Quality check needed")
        else:
            print("Quality is acceptable")
```

C. CODE EXPLANATION & FLOWCHART

- **Nested Loop Audit:** Outer loop tracks products (`i`), inner loop iterates over batches (`j`).
- **Defect Formula:** Calculates defect percentage as `defective * 100 / made` with zero check.
- **Rounding:** Uses `round(percent, 2)` for clear financial and engineering precision output.
- **Inspection Trigger:** Compares defect percentage against `limit` to flag required quality checks.



D. EXPECTED & ACTUAL OUTPUT

```
Enter number of products: 3
Enter number of batches: 3
Enter defect percentage limit: 2
Enter units produced: 12
Enter defective units: 1
Product: 1 Batch: 1
Defect percentage: 8.33 %
Quality check needed
Enter units produced:
```

E. RESULT & EXECUTION SCREENSHOT

Result: Manufacturing defect rate matrix program executed successfully and flagged non-compliant batches correctly.

